The Minute Cryptic Decrypted: An Algorithmic Approach to Solving Wordplay Puzzles

Amanda Huang, Gbemiga Salu, and Miya Zhao
Yale College
New Haven, CT 06511
amanda.huang@yale.edu, gbemiga.salu@yale.edu, miya.zhao@yale.edu

Abstract

Minute Cryptic is a crossword game that incorporates letter-play and wordplay to create linguistic puzzles. We developed a two-stage pipeline combining machine learning classification with specialized solving algorithms to automatically solve these puzzles. Our approach first classifies puzzles into three categories (anagrams, hiddens, or selectors) using logistic regression with GloVe embeddings, then applies tailored algorithms for each type. Despite limited training data (39 examples), our system achieves 69% classification accuracy. The pipeline achieved a 38% end-to-end solving accuracy; however, the correct answer was found in the candidate list 67% of the time. This work contributes to understanding how computational methods can solve pun-based puzzles and demonstrates the challenges LLMs face in capturing human nuances in communication.

1 Introduction

Created by Angas Tiernan, Minute Cryptic is a crossword game that relies on letterplay and wordplay to create linguistic puzzles. Each daily clue consists of two components: a definition (a direct hint at the answer) and the wordplay (a mechanism to build the solution through language tricks). The wordplay typically involves "fodder" (the letters being manipulated) and "indicators" (words instructing the solver how to manipulate that fodder). By identifying these indicators, a player can determine the puzzle type (anagrams, selectors, or hidden) and solve it accordingly.

The game has gained a dedicated following, with many integrating the puzzle into their daily routines. These clues often require quite the mental gymnastics, often tasking the solver with rearranging, deleting, or substituting letters to reveal the answer. Given this structure, we investigated whether a systematic, computational approach could be used to solve the more straightforward puzzle types. Prior work suggests this is a significant challenge; Zangari et al. (2025) found that Large Language Models (LLMs) often struggle to capture the semantic richness required for puns, relying on surface-level patterns rather than the nuanced reasoning humans use to interpret humor (Zangari et al., 2025). Their findings suggest that success on general benchmarks does not necessarily translate to a genuine understanding of wordplay.

In this project, we sought to bridge this gap by building a program to categorize and solve Minute Cryptic puzzles using a combination of machine learning and natural language processing. Specifically, we explored how word embeddings could aid in deriving an artificial understanding of these linguistic tricks, allowing the model to determine the final answer based on the provided definition. Success in this regard would signal a deeper grasp of human humor and sarcasm —nuances that remain notoriously difficult for AI. While our initial focus lies on direct wordplay mechanisms, we aim to eventually expand this framework to address puzzles involving a combination of tactics or nested puns.

2 Methodology

2.1 System Architecture

To address this challenge, our group developed a predictive solving system for Minute Cryptic puzzles, split into two parts: type classification followed by algorithm-based solving.

While Minute Cryptic features a wide variety of puzzle mechanics, we narrowed our analysis to "letter-play" puzzles, specifically focusing on three distinct categories: Anagrams, Hiddens, and Selectors. Our primary predictive model utilized logistic regression to classify incoming clues into one of these three types based on their linguistic features. Once classified, the system routed the clue to a specialized solving algorithm designed to

parse the wordplay and extract the correct answer.

Anagram puzzles involve scrambling letters to create a solution. **Hidden puzzles** often have the answer "hidden" among the fodder, and **selector puzzles** typically indicate selecting certain parts of the fodder to create the final solution. We then developed algorithms tailored to each puzzle type. Finally, we combined the two components to solve any clue.

2.2 Data Collection

We constructed a novel dataset of Minute Cryptic puzzles by sourcing content directly from the official website and YouTube channel. Given the niche nature of the game and the absence of existing repositories, we manually parsed over 150 puzzles, extracting the clues, indicators, fodder, definitions, and corresponding answers. To ensure compatibility with our pipeline, we applied strict inclusion criteria: puzzles were required to be strictly classified as Anagram, Hidden, or Selector types; possess a single-word solution; and rely on a single solving mechanism rather than a combination of processes. This filtration process significantly reduced the raw data, with approximately 25% of the original set meeting the requirements, yielding a final, high-quality dataset of 39 labeled examples.

To provide a robust evaluation despite the limited dataset size, we utilized K-Fold cross-validation alongside Scikit-Learn's `train_test_split` function (Kohavi, 1995; scikit-learn developers, 2025; Buitinck et al., 2013). While the sample size was small, K-Fold validation mitigated the risk of overfitting by ensuring that every data point served as both a training and testing instance across different iterations. This methodology maximized the utility of our available data, allowing for a more reliable estimate of the model's accuracy and precision than a single train-test split would permit.

2.3 Classification Model

To implement our classification system, we integrated a suite of specialized Python libraries, utilizing NumPy and pandas for data processing, Scikit-Learn for model development (scikit-learn developers, 2025; Buitinck et al., 2013), and the GloVe model for semantic embedding (Pennington et al., 2014). We specifically utilized GloVe for word vectorization due to its efficacy in capturing semantic relationships by minimizing the difference between the dot product of word vectors and the logarithm of their co-occurrence probability. These tools collectively streamlined our pipeline, covering everything from feature engineering to final model training.

For the core classification program, we developed a multinomial logistic regression model to identify the puzzle type based on features derived from the indicator, fodder, and answer length. We selected this discriminative approach over generative alternatives like Naive Bayes because it typically yields higher accuracy by establishing more complex decision boundaries that were crucial for handling our specific categorical independent variables. Using Scikit-Learn's `LogisticRegression` and `LabelEncoder`, the system processes the user's input (clue, indicator, fodder, definition, and solution length) to generate a predicted classification, accompanied by a probability distribution across the three target categories: Anagrams, Selectors, and Hiddens.

```
Enter the clue: De Niro's acting decreased with Heat
Enter the indicator: acting
Enter the fodder words: De Niro
Enter the definition: decreased with Heat
Enter the solution length: 6

----- Prediction Result -----
Predicted Category: Anagrams

Probabilities:
  Anagrams: 0.5437
  Hiddens: 0.1188
  Selectors: 0.3375
```

Figure 1: Demonstration of the classification pipeline: User imputed parameters and resulting predicted category probabilities.

2.3.1 Features

length: The total character count of the target solution. We hypothesized that longer solution lengths would correlate with specific, more complex wordplay mechanisms.

fodder_length: The length of the word(s) that will be "played with." We believed it would increase the accuracy of Anagrams if the fodder length and answer length were identical.

fodder_word_count: The number of distinct words comprising the fodder. This feature aids in distinguishing categories; for instance, hidden clues frequently rely on bigrams (two words), whereas Selectors often utilize larger word groupings.

Semantic Similarity Features (GloVe): We employed Global Vectors for Word Representation

(GloVe) (Pennington et al., 2014) to capture the semantic nuances of the indicator words. Using a pre-trained GloVe model (2 billion tweets, 50 dimensions), we first established reference "word banks" containing known indicators for each of the three puzzle types. We then calculated the average cosine similarity between the current puzzle's indicator and each of these reference banks. This process yielded three distinct semantic features (`glove_anagram`, `glove_hidden`, and `glove_selector`) allowing the model to quantify how closely a new indicator semantically resembles the established conventions of each category.

2.4 Solving Algorithms

2.4.1 Anagram

For Anagram-type puzzles, our algorithm begins by extracting the fodder and removing any non-alphabetical characters. Then, our program generates all possible permutations of these characters. Afterwards, we use a dictionary of the 100,000 most common English words to filter our results to contain only valid options. To handle the fact that many cryptic Anagrams involve an additional letter appended or pre-pended to the fodder, we implemented an augmentation step where every letter of the alphabet is added both to the start and end of each candidate permutation. After generating all possible candidates, the algorithm retains a list of plausible solutions based on our system.

2.4.2 Hidden

Hidden clues typically conceal the answer within a span of letters in the clue text. To identify these patterns, we implemented an n-gram-based approach. Given the known solution length, the solver constructs all n-grams of length n from the combined fodder text, where n is the solution length. The program takes the fodder, removes any non-alphabetical characters, and creates a string of letters. From there, we utilize a sliding-window method to enumerate every possible hidden substring. This list of n-grams is compared to a 100,000 most common English words list to filter out invalid options. Like Anagrams, we are left with a list of plausible solutions based on our algorithm.

2.4.3 Selector

Selector clues rely on positional extraction indicators like "first," "last," "even," and so on. To handle this class of puzzle, our program hard-coded selection patterns reflecting the most common cryptic constructions. As a result, the program generated a list by selecting every first letter, last letter, even or odd letters, and variants involving half-word selections by using the fodder. As with the other solvers, the candidates are then filtered using the 100,000 most common English words dictionary. This systematic selection of patterns allowed our solver to handle a wide range of indicator styles while remaining computationally light. As before, the program returns a list of all the words that passed through our programs.

2.4.4 Special Solves

To improve our solver, we expanded its logic to handle more complex puzzle types, including those that involve reversing words or swapping letters within an Anagram. For Hidden and Selector puzzles, the system looks for answers by reading the text both forwards and backwards, automatically discarding any result that isn't a valid English word. Since Anagrams are the most common type, we also built in some flexibility: instead of just rearranging the exact letters provided, our program checks if adding or removing a letter at the beginning or end reveals the correct answer. This allows us to solve tricky clues where the scrambled

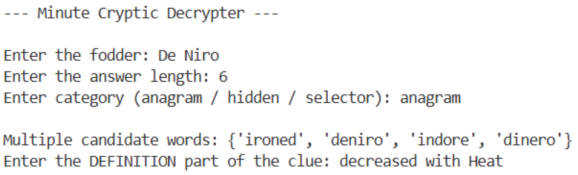

Figure 2: Candidate Generation Process: The system utilizes the selected solving algorithm to generate an exhaustive list of potential answers.

2.5 Choosing A Top Candidate

Recognizing that a raw list of candidate solutions isn't ideal, we implemented a semantic ranking mechanism using cosine similarity of the GloVe vectors. Following established computational methods for measuring word relatedness (Kruszewski and Baroni, 2015), we calculated the cosine similarity between the definition provided in the clue and each valid candidate word generated by our algorithms. The system then ranked these candidates by their semantic score, selecting the top-ranked word as the predicted answer. For definitions with multiple words, the average cosine similarity of each word in the phrase was taken as a comparison

to the candidate.

Given the limited scale of our testing data, we added a more inclusive metric for evaluation: we defined a "successful" solve as one where the correct answer appeared anywhere within our generated list of valid candidates. In cases where the algorithms failed to generate any valid English words, the system returned a "No word found" status.

```
--- GloVe Scoring ---
Definition: decreased with Heat
Best Match: ironed

ironed          0.0076
dinero          -0.0259
indore          -0.0498
deniro          -0.1957

Final Answer: ironed
```

Figure 3: Final Answer Selection: Ranking valid candidates via GloVe cosine similarity to identify the most probable solution.

3 Results

3.1 Overall Performance

	Category Cracked	Word Found? (told category)	Correct Final Answer	Passes All Parts	Total
Selector	12	9	2	2	14
Hidden	8	10	7	6	10
Anagram	15	7	6	6	15
Overall	35	26	15	14	39

Figure 4: Overall Performance: Frequency of successful outcomes across the pipeline: Classification, Solving, Semantic Ranking, and Overall Accuracy by category.

	Category Cracked	Word Found? (told category)	Correct Final Answer	Passes All Parts
Selector	85.71%	64.29%	14.29%	14.29%
Hidden	80.00%	100.00%	70.00%	60.00%
Anagram	100.00%	46.67%	40.00%	40.00%
Overall	89.74%	66.67%	38.46%	35.90%

Figure 5: Overall Performance: Percentage of successful outcomes across the pipeline: Classification, Solving, Semantic Ranking, and Overall Accuracy by category.

To identify limiting factors within our pipeline, we tracked the success rate at each distinct stage of the solving process. While the system achieved an overall end-to-end solve rate of approximately 38%, analyzing the specific drop-off points highlights clear areas for improvement. We noted that the initial category prediction rate appeared artificially high, likely because the preliminary evaluation included data points that were also used during model training. Consequently, to obtain a more accurate measure of the model's true performance, we relied on a 5-fold cross-validated classification report. This rigorous analysis revealed a clear hierarchy in performance: Anagrams proved the most robust, achieving a perfect 100% recall. Conversely, the model struggled to distinguish between Hiddens and Selectors, frequently confusing the two categories. Hiddens performed the worst overall, suggesting that our current linguistic features may not fully capture the nuanced indicators specific to that puzzle type.

```
----- Classification Report (aggregated over folds) -----
              precision    recall  f1-score   support

    Anagrams       0.88      1.00      0.94        15
     Hiddens       0.44      0.40      0.42        10
   Selectors       0.62      0.57      0.59        14

    accuracy                           0.69        39
   macro avg       0.65      0.66      0.65        39
weighted avg       0.67      0.69      0.68        39


-----Confusion Matrix (aggregated over folds) -----
[[15  0  0]
 [ 1  4  5]
 [ 1  5  8]]
```

Figure 6: Confusion Matrix and Classification Report of Cross-Validated Data

Further analysis of the misclassified instances reveals a tight margin between Hidden and Selector puzzle types. In many cases, the predicted probabilities for these two categories fell within a narrow 10% margin of one another. As illustrated by the five misclassified examples in Figure 7, the distinction was often negligible; in the most extreme case, the difference in probability between a Hidden and a Selector classification was as low as 0.87%. This suggests that our current feature set may not yet fully capture the subtle linguistic boundaries separating these two mechanics, leading the model to "guess" with low confidence rather than making a definitive prediction but we're close.

Clue	True	Predicted	P(Anag)	P(Hidd)	P(Selec)
Provide a lipstick sample for model	Hiddens	Selectors	0.52%	26.94%	72.54%
Go thirds in pet turkey	Selectors	Hiddens	4.64%	48.11%	47.24%
Original amigos each only halfway through paperwork	Selectors	Hiddens	22.06%	44.39%	33.55%
Inside Stephanie's revealing skirt	Hiddens	Selectors	15.75%	41.68%	42.57%
Murder ingredient found in microwave?	Hiddens	Selectors	7.15%	43.08%	49.76%

Figure 7: Misclassified examples showing close probability distributions. P(A) = P(Anagram), P(H) = P(Hidden), P(S) = P(Selector).

Most importantly, we were looking at overall performance, and at 69% accuracy, we considered it a passing score.

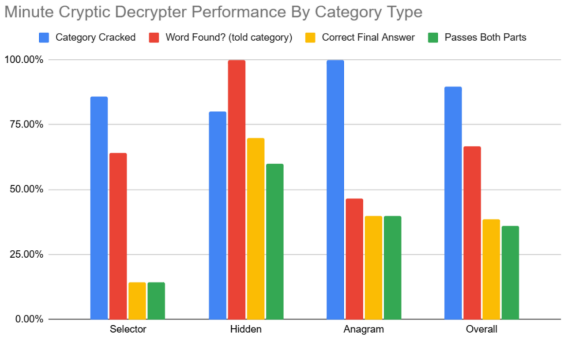

Figure 8: Overall Performance (Accuracy) for Each Category at Each Step of the Algorithm

4 Discussion

Interestingly, each category had a different limiting step, leading to our overall low solve rate of 38%.

Critically, our analysis identified the semantic ranking component as the primary bottleneck for Selector puzzles. While the algorithm successfully generated the correct candidate solution in 64% of cases, the cosine similarity function frequently failed to rank it as the top choice. This underperformance is likely attributable to the structure of Selector definitions, which often consist of complex, multi-word phrases. Averaging the word vectors for these longer strings may have caused a "dilution" of semantic specificity, obscuring the precise relationship between the definition and the target word.

Hidden puzzles were limited by categorization and semantics. When provided with the correct category, the solving algorithm achieved a perfect 100% success rate in retrieving the valid answer. However, this strength was often neutralized by the model's poor performance in the initial classification stage. Notably, despite these classification struggles, the semantic ranking module performed best on Hidden puzzles, suggesting that once the correct candidates are generated, the semantic link between a Hidden clue's definition and its answer is significantly more direct and measurable than in other categories.

Anagrams presented a unique problem. While our model was excellent at identifying them (with essentially 100% recall in the model trained on all the data), it struggled to actually find the solution. Unlike the other categories where the answer was usually found but ranked poorly, with Anagrams, our algorithm frequently failed to generate the correct word at all. Even though we tried to make the solver flexible (allowing it to add or remove letters to catch tricky clues) it often couldn't construct the right answer from the scrambled letters. As a result, the correct solution never even made it to the ranking stage.

Interestingly, we had one case where the wrong category was predicted (Selector as opposed to Hidden), but the correct answer was still found in the clue "Provide a lipstick sample for model," motivating our distinction between passing both parts (the solver and the categorizer) and just finding the correct answer.

On average, our algorithms generated a list of about six possible words for each clue. This meant that to solve the puzzle, the system had to correctly identify the one true answer from that group. When we looked at the cases where the correct word was successfully generated (about 20 instances), our ranking method proved highly effective. It placed the correct answer in the #1 spot 55% of the time, and within the top two choices 80% of the time. These results confirm that using cosine similarity is a reliable way to sort through potential answers based on their meaning. This success is particularly encouraging for our future work; because the system proved it can understand the semantic link between a definition and an answer, we believe we can expand this approach to solve much harder puzzles (like nested puns or clues requiring word substitution) where simple letter-matching isn't enough.

5 Limitations

It is important to note that the final performance statistics presented in this section are derived from a model trained on our complete dataset. Given the limited size of our corpus (39 puzzles), utilizing the full dataset allowed us to maximize the number of examples available to test the efficacy of our solving algorithms and ranking logic. We acknowledge that testing on training data introduces bias, and future iterations of this project will prioritize validation against completely unseen cases. Consequently, when evaluating the categorization step specifically, the cross-validated confusion matrix provides a more accurate and rigorous measure of performance than the heatmap generated by the full model, as the latter reflects results from data the model had already encountered during training.

The primary limitation of our work is the small dataset size of only 39 labeled examples, which constrains our findings. Additionally, we limited

our scope to single-word solutions and excluded combination puzzles, which represent a significant portion of Minute Cryptic puzzles. The cosine similarity ranking method, while effective overall, showed weakness in handling multi-word definitions, particularly for Selector puzzles. Future work could explore larger datasets, more sophisticated semantic similarity measures, and extension to multi-word and combination puzzle types.

In our group discussions, we explored a range of additional features derived from the clues' text, indicator/fodder structure, and definitional properties in an effort to improve classification performance across clue types. These additional features had linguistic motivations behind them. For instance, we considered part of speech tags for indicators, vowel-consonant ratios in the fodder, indicator-fodder positional relationships, and punctuation based cues. Ultimately, additional features introduced challenges when combined with our L2-regularized model.

As the feature space expanded, the model became increasingly sensitive to the regularization penalty, leading to coefficient shrinkage. This reduced our features' effective influence on predictions. Since L2 regularization does not perform feature selection, some weak or redundant features remained in our model with small but nonzero weights, increasing model complexity without having corresponding gains in generalization. This was especially problematic given the relatively limited size of our dataset. In this case, additional features exacerbated multicollinearity (Chan et al., 2022) and introduced variance rather than signal.

Eventually, we found that a more restrained feature set focused on fodder length, word counts, and GloVe-based indicator similarity produced more stable and interpretable results under L2 regularization than perhaps a broader, more expressive feature representation would have.

6 Linguistic Assumptions

For this project, we assume that the structure of cryptic clues is formulaic, typically consisting of an indicator, a fodder segment, and a definition, all of which are separate from each other. This assumption allows for the framing of solving as a classification and transformation problem, where identifying the indicator and fodder is sufficient to generate candidate answers. In reality, cryptic clue structure is often more fluid and overlapping. Individual words can simultaneously function as indicator and fodder, or as fodder and definition, depending on the intended reading of the clue. Additionally, some clues distribute their cryptic instructions across multiple words, challenging the notion of clearly separable structural roles. While the formulaic assumption captures a large subset of common cryptic constructions, it abstracts away from the full flexibility of cryptic crossword language. two

This project also assumes a semantic simplicity of our clue definitions. We implicitly treat definitions as relatively direct lexical mappings to the solution, using cosine similarity on GloVe embeddings to categorize similarities between our definitions and solutions. This assumes of course that the definition corresponds straightforwardly to a standard dictionary-like definition. In practice, however, many cryptic clues have definitions that require deeper semantic interpretation, inference, or figurative reasoning. This is evident in clues that end with a question mark, where the definition may rely on humor, irony, or indirect association instead of a literal meaning. By not explicitly modeling this sort of complexity, our solver prioritizes surface-level lexical cues over richer interpretations, which can limit its ability to handle more playful or non-literal definitions.

A third assumption is that lexical resources derived from standard English are sufficient for modeling and solving cryptic clues. In particular, parts of our solution algorithms rely on a fixed dictionary consisting of the 100,000 most common English words to validate candidate solutions and guide feature extraction. This assumption simplifies the implementation, allows us to capture variations of words that aren't present in a more standard dictionary (such as plurals), but it overlooks the fact that cryptic crosswords frequently draw on a more eclectic vocabulary. Proper nouns, archaic forms, abbreviations, specialized jargon, and less common terms all appear in cryptic clue solutions. As a result, our solver may fail to recognize valid answers that fall outside this restricted word bank.

References

Lars Buitinck, Gilles Louppe, Mathieu Blondel, Fabian Pedregosa, Andreas Mueller, Olivier Grisel, Vlad Niculae, Peter Prettenhofer, Alexandre Gramfort, Jaques Grobler, Robert Layton, Jake VanderPlas, Arnaud Joly, Brian Holt, and Gaël Varoquaux. 2013. Api design for machine learning software: expe-

riences from the scikit-learn project. In *European Conference on Machine Learning and Principles and Practices of Knowledge Discovery in Databases (ECML PKDD)*. Also available from HAL: hal-00856511.

Jireh Yi-Le Chan, Steven Mun Hong Leow, Khean Thye Bea, Wai Khuen Cheng, Seuk Wai Phoong, Zeng-Wei Hong, and Yen-Lin Chen. 2022. Mitigating the multicollinearity problem and its machine learning approach: A review. *Mathematics*, 10(8):1283.

Ron Kohavi. 1995. A study of cross-validation and bootstrap for accuracy estimation and model selection. In *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI 1995)*, pages 1137–1145.

Germán Kruszewski and Marco Baroni. 2015. So similar and yet incompatible: Toward the automated identification of semantically compatible words. In *Proceedings of the 2015 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL HLT)*, pages 964–969.

Jeffrey Pennington, Richard Socher, and Christopher D. Manning. 2014. Glove: Global vectors for word representation. *Empirical Methods in Natural Language Processing (EMNLP) 2014*, pages 1532–1543.

scikit-learn developers. 2025. scikit-learn documentation (stable), version 1.8.0. `https://scikit-learn.org/stable/index.html`. Accessed December 12, 2025.

Alessandro Zangari, Matteo Marcuzzo, Andrea Albarelli, Mohammad Taher Pilehvar, and Jose Camacho-Collados. 2025. Pun unintended: Llms and the illusion of humor understanding. *arXiv preprint*.